OMNIVERSAL ARCHITECT LABS — CURRENCY LABORATORY

Pilot Packet 001 — Sealed Reference Execution Report

Executable reference model, results, assertion testing, findings about the packet itself, and a numerical check of the M04 feedback claims

Document ID	OAL-CURR-PPLX-PILOT-001-REF (qualified ID: perplexity/pilot-packet-001-ref-v0.1)
Version / date	Version 0.1 — November 7, 2026
Prepared by	Perplexity, member of the OAL AI collaboration team
Generation platform	Perplexity Computer; underlying model for this session reported by the platform as Claude Opus 5.5 (Anthropic)
Status	PROPOSED. Pre-adoption reference execution. Not an owner-adopted laboratory run. Pre-canon.
Distribution	SEALED. Recommended handling: give this report to other analysts only after they have submitted first-pass run records for Pilot Packet 001. Circulating it earlier would make their runs non-blind.
Evidence level	E2 EXECUTED for program outputs; E1 DERIVED for the analysis in Sections 5–7. Nothing here is E3; nothing here shows that any real Quantum Credit capability exists.
Companion documents	OAL-CURR-PPLX-PILOT-001 (the packet this report executes); OAL-CURR-PPLX-BOUNDARY-001
Accompanying file	oal_pilot001_reference_model.py — SHA-256 5588680542a98605c60f2bd846504aa76415359fe0281fd86f07c827b00eba9a

1. Why this report exists, and why it is sealed

My recommendation listed “implement a minimal executable model” as a P1 action, with success defined as identical inputs reproducing identical outputs and deliberately defective implementations failing the relevant checks. This report is that model. I wrote it while drafting Pilot Packet 001 so that the packet would be tested against something stricter than prose before anyone else spent effort on it.

Because it contains results, it would spoil a blind first pass. I recommend the owner hold it back until the other members have submitted their own records, then release it as a comparison point. It is not a reference answer key. If another member’s careful record disagrees with mine, the disagreement is evidence, and my implementation is as likely to be wrong as theirs.

2. What was executed

A single Python file (Python 3.14.8, standard library only) implements fixture F-001, the seven configurations, the shared definition and coherence checks (rule R-GATE), the request-identity rule (R-ID), the completion predicate CP-001, and two verdicts: Verdict A (internal consistency) and Verdict B (requested outcome). It runs all eleven cases against all seven configurations — 77 case–configuration pairs, and 91 run steps because the two replay cases each submit two requests. It then runs six deliberately broken implementations against the consistency checks, and finally computes the M04 feedback recurrence for nine values of α over 150 periods.

The run is deterministic. Running the file again produces the same output. The full machine-readable results are written to pilot001_reference_results.json, and the console log to run_log.txt; both are reproduced in summary in the tables at the end of this report, and the code is listed in full in Appendix A.

How the consistency checks work

Verdict A is computed by checks that compare the state before and after a step with what the configuration's declared capability allows. Every configuration must keep the definitional identity (total equals A plus B), must report a delivery count that matches B's actual change, and must not change anything when it reports that nothing was delivered. Transfer-type configurations must leave the total unchanged and must debit A by exactly what B received. Creation-type configurations must leave A untouched and raise the total by exactly what was created. The evaluation-only configuration must not change the world at all. One further check runs across steps rather than within one: a request ID may reach DELIVERED at most once.

3. Results, in prose

Table R1 at the end gives every result. The pattern is as follows.

Baseline. C-EVAL returns a value description and changes nothing; Verdict A is consistent and Verdict B is not satisfied. C-XFER, C-XFER-NA, and C-ENACT-S all leave A at 7, B at 3, and the total at 10. C-CREATE and C-ENACT-M leave A at 10, B at 3, and the total at 13. C-ENACT-U returns UNDERSPECIFIED_CONFIG and changes nothing. All seven are internally consistent.

Insufficient source (A holds 2). Both transfer configurations return REJECTED_INSUFFICIENT. C-ENACT-S, whose target state includes "A has lost 3," returns INCOHERENT_TARGET instead, because holdings cannot be negative by definition. C-CREATE and C-ENACT-M deliver normally, leaving the total at 5.

Replay. Every delivering configuration returns DUPLICATE_SUPPRESSED on the second submission of REQ-1 and changes nothing. When the second request has the same content but a new ID (REQ-2), every delivering configuration delivers again: transfer configurations leave A at 4 and B at 6; creation configurations leave B at 6 and the total at 16.

Interrupted delivery. The atomic configurations (C-XFER, C-ENACT-S) roll back to the starting state with FAILED_ROLLED_BACK. C-XFER-NA leaves A at 8 and B at 2 with PARTIAL. C-CREATE and C-ENACT-M leave B at 2 and the total at 12 with PARTIAL. No configuration reports DELIVERED, and none satisfies CP-001.

Consent revoked. Configurations that take units from A (C-XFER, C-XFER-NA, C-ENACT-S) return REJECTED_UNAUTHORIZED. C-CREATE and C-ENACT-M deliver anyway, because A's consent governs only A's units and the packet declares no authority over creation.

Undefined and incoherent requests. Every configuration returns UNDEFINED_TARGET for the unstated unit type, the undeclared type Q, and the unstated quantity; every configuration returns INCOHERENT_TARGET for both contradiction cases. None of them changes anything.

4. Assertion testing: did the checks catch broken implementations?

A set of checks that cannot detect a broken implementation is not checking anything. I wrote six deliberately defective implementations, each of which reports success while doing something wrong: a transfer that credits B without debiting A (MUT-1); a transfer that certifies delivery when a fault stopped it after two units (MUT-2); a transfer that, when A is short, quietly fills the gap from a pool the fixture does not contain (MUT-3); an evaluation that moves a unit (MUT-4); a creation that also secretly debits A (MUT-5); and a transfer engine with the replay guard removed, so that the same request ID delivers twice (MUT-6).

My first draft of the checks missed one of them. In that draft some checks were attached to configuration labels ("if the configuration is called C-CREATE, check that A did not change") rather than to the capability the configuration declared. The creation mutant carried a different label, so the creation checks never ran on it, and it passed as consistent. I corrected the checks so that each attaches to the declared capability. The script keeps a fully label-bound mode so the hazard can be reproduced: in that mode the checks miss three of six mutants (MUT-1, MUT-3, MUT-5). With capability-bound checks, all six are caught. Table R2 records both.

MUT-6 deserves a separate note. Each of its steps passes every within-step check; the world after each step is perfectly consistent. Only the across-step rule (a request ID may reach DELIVERED once) catches it. A run record that checks each step in isolation will not see replay faults.

The general lesson, which I think applies to every member who writes an implementation: bind every invariant to the declared capability contract, not to a name; and include at least one check that looks across the whole log.

5. Findings

Each finding states its evidence level. None of them is a finding about whether Quantum Credits can do anything outside this model.

Finding 1 — Minimal enactment collapses into creation in this fixture (E2, E1)

C-ENACT-M and C-CREATE produce identical observable states in all eleven cases. The only difference is the origin label on B's new units — "enacted" versus "minted" — and I wrote those labels into the configurations. In this fixture, enactment under a minimal-change rule is observationally indistinguishable from creation. A test that needs to tell them apart would need an observable that differs between them, and the packet has none. This is the clearest example of the risk I raised in my recommendation: transfer and enactment, or creation and enactment, can look identical.

Finding 2 — The same shortfall gets two different barrier classifications (E2, E1)

C-XFER and C-ENACT-S leave the world in the same state in every case. But when A holds only 2 units, transfer reports REJECTED_INSUFFICIENT (a capability limit: not enough inventory) while enactment-with-named-source reports INCOHERENT_TARGET (a definitional impossibility: A cannot hold −1). Nothing about the world differs; what differs is whether the source's loss is part of the requested target. This matters for the boundary work. Whether a barrier is classed as a capability limit or as incoherence can depend on how the

target is written, not on the situation. Boundary taxonomies should record the target formulation alongside the classification.

Finding 3 — Evaluation-only behaves as defined; the interface keeps the verdicts apart (E0, E2)

C-EVAL is consistent in every case and never satisfies CP-001. That is its definition, not a discovery. What the run does show is that the reporting interface can record "consistent but did not deliver" without calling the configuration contradictory — the outcome ChatGPT's two-verdict structure was designed to protect.

Finding 4 — Revoking consent discriminates only source-dependent configurations (E2), exposing gap G-1

P001-F01-d separates configurations that take from A from those that do not. It says nothing about creation, because the packet defines no authority over creation (rule R-AUTH). The packet left this gap open on purpose, but the run shows its effect concretely: under v0.1, creation delivers even when every existing party has refused. Whether that is acceptable is a design question for the owner.

Finding 5 — Replay protection depends entirely on what counts as "the same request" (E2), gap G-2

Under R-ID, a repeated request ID is suppressed and a new ID with identical content is delivered again. Both results follow from the rule; neither shows the rule is right. If the owner's intent is that the obligation "B should have 3" can be satisfied only once, R-ID is the wrong rule and P001-F01-b2 should be suppressed too. That would require the laboratory to define obligation identity, not just request identity. This bears directly on 1COIN, which several members describe as an instrument for a resolved obligation.

Finding 6 — Partial-delivery outcomes are fixed by stipulation (E0), gap G-3

Every interrupted-delivery result follows from the atomicity rule I declared. Atomic configurations roll back; non-atomic ones leave a partial state; creation leaves two orphaned units and a total of 12. A different stipulation would give different numbers. The informative part is that no configuration reports DELIVERED after a fault — the property that a premature completion certificate (MUT-2) would violate.

Finding 7 — Bare enactment is not testable (E1)

C-ENACT-U returns UNDERSPECIFIED_CONFIG in every defined case. "QC makes the outcome happen" does not, by itself, say what else changes. Every runnable enactment configuration needed an extra rule, and once that rule was added, enactment matched either transfer (Finding 2) or creation (Finding 1) in this fixture. I do not think this shows enactment is empty. It shows that the earlier documents' enactment language needs a side-effect rule before it can be compared with anything.

Finding 8 — F03 tests the reporting layer, not the capability layer (E2, E1)

All seven configurations return the same status for every F03 case, because the definition and coherence checks run before any capability does. F03 therefore cannot discriminate capability configurations; it confirms that "refers to nothing" and "cannot all be true" come out differently. P001-F03-I2 also confirms that the definitional identity binds every configuration, including creation and enactment: "B gains 3, A stays at 10, and the total stays at 10" is incoherent even for a configuration that is allowed to break conservation.

Finding 9 — Implementation checks must bind to capability, not to labels (E2)

Described in Section 4.

6. Defects found in the packet while building this model

Building the model forced several decisions that my first outline of the packet had not made. I fixed these before releasing Pilot Packet 001 v0.1, which is why the packet is cleaner than my recommendation was. I list them because the fixes came from my implementation, not from any other member, and because they show what a first-pass analyst would otherwise have had to invent.

- **No request-identity rule.** The replay case could not be run until "the same request" was defined. Added as R-ID; P001-F01-b2 was added to expose the choice.
- **No atomicity rule.** The interrupted-delivery case had no defined outcome. Added as R-ATOM, with C-XFER-NA as an explicit non-atomic variant.
- **No unit identity or origin labels.** Without them, creation and transfer could not be told apart even in principle. Added to fixture F-001.
- **Conservation and the definitional identity were not separated.** My outline treated "total stays at 10" as a property to check in every configuration, which would have counted creation as inconsistent by fiat — exactly the move I criticized in my recommendation. The fixture now separates the two, and P001-F03-I2 tests the difference.
- **Enactment had one configuration.** It is now three (unspecified, minimal-change, named-source), because the single version could not be run.
- **No order of checks.** Without R-GATE, an analyst could have run a creation step on an undefined request. Added.

Two gaps remain open on purpose: who authorizes creation (G-1) and whether the obligation rather than the request should be the unit of replay protection (G-2). Both are owner decisions.

7. Numerical check of the M04 feedback claims

Several members analyzed the feedback system $I = 2G$, $G = B + \alpha I$ in the latest round. M04 is a Cycle 3 topic and not part of Pilot 001, but since the claims were made in documents that will be published, I checked them numerically with $B = 100$ and the starting value $I_0 = 2B = 200$ used in Gemini's closed form. Table R3 gives the values.

- **Fixed point.** The simultaneous equations give $I = 2B/(1 - 2\alpha)$. With $B = 100$: $\alpha = 0$ gives 200, $\alpha = 0.25$ gives 400, $\alpha = 0.4$ gives 1,000. At $\alpha = 0.5$ there is no finite solution. For α above 0.5 the algebraic solution is negative (for example −400 at $\alpha = 0.75$), which is inadmissible if allocations must be non-negative. ChatGPT, Claude, Gemini, and my earlier response all state this correctly.
- **Lagged recurrence.** For $I(t+1) = 2B + 2\alpha \cdot I(t)$, the run confirms Gemini's closed form, including linear growth at $\alpha = 0.5$: $I(150) = 30{,}200 = 200 \times 151$. It confirms Claude's observation that at $\alpha = -0.5$ the fixed point is 100 but the sequence alternates between 200 and 0 without settling.
- **A point not raised by anyone, including me.** Convergence in the limit is not convergence within the horizon. At $\alpha = 0.49$ the sequence converges in principle to 10,000, but after 150 periods it has reached only 9,526.7 — still 4.7 percent short. For the 150-year scenario, "stable" should be defined as reaching a stated tolerance within the horizon, not as satisfying $|2\alpha| < 1$.
- **A qualification of Gemini's wording.** Gemini wrote that the $\alpha = 0.5$ singularity "is an artifact of simultaneous static resolution" that lagged evaluation avoids. The run supports the arithmetic but not the reassurance. With lag, $\alpha = 0.5$ no longer divides by zero, but it still has no steady state; it grows without bound, more slowly. Lag changes the failure from nonexistence to unbounded linear growth. Whether that is acceptable over 150 years is a design judgment, not something the lag resolves.

- **A second qualification.** Gemini listed “conservation balance identities (QC + NCOIN = 0)” among things that can be established analytically. They can be checked analytically only after QC and NCOIN quantities are defined in a common unit with a stated boundary. ChatGPT made the same point. Until then there is nothing to check.

8. Limitations

- The model implements my reading of my own packet. Another analyst might read a rule differently, and that difference would be a legitimate finding about the packet.
- The configurations are toy models. They show what the stated rules imply, nothing more.
- The mutation suite has six mutants. Passing it does not prove the checks are complete; it shows they catch six named errors.
- The underlying model of this session is shared with another collaborator (Claude). A Claude implementation that agrees with this one is weaker corroboration than agreement between unrelated models.

9. How to reproduce

Save Appendix A (or the accompanying .py file) as oal_pilot001_reference_model.py and run `python3 oal_pilot001_reference_model.py`. No packages are needed beyond the Python standard library. The script prints the run log and writes pilot001_reference_results.json. Compare the SHA-256 of the script with the value in the document-control table to confirm you are running the same file. Run outputs may differ in their hash if Python prints floating-point numbers differently, but the values should agree.

Tables and Reference Records

Table R1 — Full results (E2 EXECUTED)

A/B/T show holdings of A, holdings of B, and total, before → after. CP shows whether completion predicate CP-001 was met. VA is Verdict A. For the two replay cases, both steps are listed.

Case	Config	Status	A	B	T	CP	VA
P001-F01-BASE	C-EVAL	VALUED	10→10	0→0	10→10	No	C
P001-F01-BASE	C-XFER	DELIVERED	10→7	0→3	10→10	Yes	C
P001-F01-BASE	C-XFER-NA	DELIVERED	10→7	0→3	10→10	Yes	C
P001-F01-BASE	C-CREATE	DELIVERED	10→10	0→3	10→13	Yes	C
P001-F01-BASE	C-ENACT-U	UNDERSPECIFIED_CONFIG	10→10	0→0	10→10	No	C
P001-F01-BASE	C-ENACT-M	DELIVERED	10→10	0→3	10→13	Yes	C
P001-F01-BASE	C-ENACT-S	DELIVERED	10→7	0→3	10→10	Yes	C
P001-F01-a	C-EVAL	VALUED	2→2	0→0	2→2	No	C
P001-F01-a	C-XFER	REJECTED_INSUFFICIENT	2→2	0→0	2→2	No	C
P001-F01-a	C-XFER-NA	REJECTED_INSUFFICIENT	2→2	0→0	2→2	No	C
P001-F01-a	C-CREATE	DELIVERED	2→2	0→3	2→5	Yes	C
P001-F01-a	C-ENACT-U	UNDERSPECIFIED_CONFIG	2→2	0→0	2→2	No	C
P001-F01-a	C-ENACT-M	DELIVERED	2→2	0→3	2→5	Yes	C
P001-F01-a	C-ENACT-S	INCOHERENT_TARGET	2→2	0→0	2→2	No	C
P001-F01-b (step 1)	C-EVAL	VALUED	10→10	0→0	10→10	No	C
P001-F01-b (step 2)	C-EVAL	VALUED	10→10	0→0	10→10	No	C
P001-F01-b (step 1)	C-XFER	DELIVERED	10→7	0→3	10→10	Yes	C
P001-F01-b (step 2)	C-XFER	DUPLICATE_SUPPRESSED	7→7	3→3	10→10	No	C
P001-F01-b (step 1)	C-XFER-NA	DELIVERED	10→7	0→3	10→10	Yes	C
P001-F01-b (step 2)	C-XFER-NA	DUPLICATE_SUPPRESSED	7→7	3→3	10→10	No	C
P001-F01-b (step 1)	C-CREATE	DELIVERED	10→10	0→3	10→13	Yes	C
P001-F01-b (step 2)	C-CREATE	DUPLICATE_SUPPRESSED	10→10	3→3	13→13	No	C
P001-F01-b (step 1)	C-ENACT-U	UNDERSPECIFIED_CONFIG	10→10	0→0	10→10	No	C
P001-F01-b (step 2)	C-ENACT-U	UNDERSPECIFIED_CONFIG	10→10	0→0	10→10	No	C
P001-F01-b (step 1)	C-ENACT-M	DELIVERED	10→10	0→3	10→13	Yes	C
P001-F01-b (step 2)	C-ENACT-M	DUPLICATE_SUPPRESSED	10→10	3→3	13→13	No	C
P001-F01-b (step 1)	C-ENACT-S	DELIVERED	10→7	0→3	10→10	Yes	C
P001-F01-b (step 2)	C-ENACT-S	DUPLICATE_SUPPRESSED	7→7	3→3	10→10	No	C
P001-F01-b2 (step 1)	C-EVAL	VALUED	10→10	0→0	10→10	No	C
P001-F01-b2 (step 2)	C-EVAL	VALUED	10→10	0→0	10→10	No	C
P001-F01-b2 (step 1)	C-XFER	DELIVERED	10→7	0→3	10→10	Yes	C
P001-F01-b2 (step 2)	C-XFER	DELIVERED	7→4	3→6	10→10	Yes	C
P001-F01-b2 (step 1)	C-XFER-NA	DELIVERED	10→7	0→3	10→10	Yes	C

Case	Config	Status	A	B	T	CP	VA
P001-F01-b2 (step 2)	C-XFER-NA	DELIVERED	7→4	3→6	10→10	Yes	C
P001-F01-b2 (step 1)	C-CREATE	DELIVERED	10→10	0→3	10→13	Yes	C
P001-F01-b2 (step 2)	C-CREATE	DELIVERED	10→10	3→6	13→16	Yes	C
P001-F01-b2 (step 1)	C-ENACT-U	UNDERSPECIFIED_CONFIG	10→10	0→0	10→10	No	C
P001-F01-b2 (step 2)	C-ENACT-U	UNDERSPECIFIED_CONFIG	10→10	0→0	10→10	No	C
P001-F01-b2 (step 1)	C-ENACT-M	DELIVERED	10→10	0→3	10→13	Yes	C
P001-F01-b2 (step 2)	C-ENACT-M	DELIVERED	10→10	3→6	13→16	Yes	C
P001-F01-b2 (step 1)	C-ENACT-S	DELIVERED	10→7	0→3	10→10	Yes	C
P001-F01-b2 (step 2)	C-ENACT-S	DELIVERED	7→4	3→6	10→10	Yes	C
P001-F01-c	C-EVAL	VALUED	10→10	0→0	10→10	No	C
P001-F01-c	C-XFER	FAILED_ROLLED_BACK	10→10	0→0	10→10	No	C
P001-F01-c	C-XFER-NA	PARTIAL	10→8	0→2	10→10	No	C
P001-F01-c	C-CREATE	PARTIAL	10→10	0→2	10→12	No	C
P001-F01-c	C-ENACT-U	UNDERSPECIFIED_CONFIG	10→10	0→0	10→10	No	C
P001-F01-c	C-ENACT-M	PARTIAL	10→10	0→2	10→12	No	C
P001-F01-c	C-ENACT-S	FAILED_ROLLED_BACK	10→10	0→0	10→10	No	C
P001-F01-d	C-EVAL	VALUED	10→10	0→0	10→10	No	C
P001-F01-d	C-XFER	REJECTED_UNAUTHORIZED	10→10	0→0	10→10	No	C
P001-F01-d	C-XFER-NA	REJECTED_UNAUTHORIZED	10→10	0→0	10→10	No	C
P001-F01-d	C-CREATE	DELIVERED	10→10	0→3	10→13	Yes	C
P001-F01-d	C-ENACT-U	UNDERSPECIFIED_CONFIG	10→10	0→0	10→10	No	C
P001-F01-d	C-ENACT-M	DELIVERED	10→10	0→3	10→13	Yes	C
P001-F01-d	C-ENACT-S	REJECTED_UNAUTHORIZED	10→10	0→0	10→10	No	C
P001-F01-e	C-EVAL	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F01-e	C-XFER	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F01-e	C-XFER-NA	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F01-e	C-CREATE	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F01-e	C-ENACT-U	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F01-e	C-ENACT-M	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F01-e	C-ENACT-S	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U1	C-EVAL	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U1	C-XFER	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U1	C-XFER-NA	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U1	C-CREATE	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U1	C-ENACT-U	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U1	C-ENACT-M	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U1	C-ENACT-S	UNDEFINED_TARGET	10→10	0→0	10→10	No	C

Case	Config	Status	A	B	T	CP	VA
P001-F03-U2	C-EVAL	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U2	C-XFER	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U2	C-XFER-NA	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U2	C-CREATE	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U2	C-ENACT-U	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U2	C-ENACT-M	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-U2	C-ENACT-S	UNDEFINED_TARGET	10→10	0→0	10→10	No	C
P001-F03-I1	C-EVAL	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I1	C-XFER	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I1	C-XFER-NA	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I1	C-CREATE	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I1	C-ENACT-U	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I1	C-ENACT-M	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I1	C-ENACT-S	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I2	C-EVAL	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I2	C-XFER	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I2	C-XFER-NA	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I2	C-CREATE	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I2	C-ENACT-U	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I2	C-ENACT-M	INCOHERENT_TARGET	10→10	0→0	10→10	No	C
P001-F03-I2	C-ENACT-S	INCOHERENT_TARGET	10→10	0→0	10→10	No	C

VA: C = consistent with declared rules; I = inconsistent. Every pair in the corrected harness is consistent.

Table R2 — Mutation suite

Mutant	Reported status	Caught — label-bound checks	Caught — capability-bound checks	What caught it (corrected checks)
MUT-1 transfer-without-debit	DELIVERED	No	Yes	transfer changed the total; A's debit does not equal B's credit
MUT-2 certify-before-delivery	DELIVERED	Yes	Yes	reported delivery does not match B's change; A's debit does not equal B's credit
MUT-3 hidden-third-party-pool	DELIVERED	No	Yes	transfer changed the total; A's debit does not equal B's credit
MUT-4 evaluation-that-moves	VALUED	Yes	Yes	state changed although status reports no delivery; evaluation-only config moved resources
MUT-5 creation-that-debits-A	DELIVERED	No	Yes	creation/minimal enactment changed A; total did not rise by the created amount
MUT-6 replay-guard-removed	DELIVERED then DELIVERED	Yes	Yes	request id(s) delivered more than once: ['REQ-1']

Table R3 — M04 feedback recurrence, B = 100, I_0 = 200

α	Fixed point 2B/(1−2α)	I(1)	I(10)	I(149)	I(150)
0.0	200.0	200.0	200.0	200.0	200.0
0.25	400.0	300.0	399.8	400.0	400.0
0.4	1,000.0	360.0	914.1	1,000.0	1,000.0
0.49	10,000.0	396.0	1,992.7	9,517.0	9,526.7
0.5	none (no finite solution)	400.0	2,200.0	30,000.0	30,200.0
0.51	-10,000.0	404.0	2,433.7	184,996.0	188,895.9
0.75	-400.0	500.0	34,199.0	1.037e+29	1.555e+29
1.0	-200.0	600.0	409,400.0	2.854e+47	5.709e+47
-0.5	100.0	0.0	200.0	0.0	200.0

Negative fixed points (α > 0.5) are algebraic solutions that are inadmissible if allocations must be non-negative. At α = −0.5 the sequence alternates 200, 0, 200, … around the fixed point of 100.

Table R4 — Files

File	Role	SHA-256
oal_pilot001_reference_model.py	Reference implementation (Appendix A)	5588680542a98605c60f2bd846504aa76415359fe0281fd86f07c827b00eba9a
run_log.txt	Console output of the run reported here	eb2c7c1c5cf5e237bf81abb61b49f5c9f3590c3f8b5fcd806640605361d4a25c
pilot001_reference_results.json	Machine-readable results	ef22b2ec69ff6dd23afcd3b65f2b294af3741b56816a90b2782f220a933553ad

Appendix A — Reference implementation source

```
"""
OAL Currency Laboratory - Pilot Packet 001 reference model
Namespace: perplexity/pilot-packet-001-v0.1
Prepared by: Perplexity Computer (session orchestrator model reported by platform: Claude Opus 5.5)
Status: PROPOSED / PRE-ADOPTION REFERENCE EXECUTION. Not an owner-adopted laboratory run.


Evidence level of every output produced by this file: E2-EXECUTED (the program ran).
An E2 result shows what the stated rules imply when executed. It is NOT evidence that any
real Quantum Credit capability exists (that would require E3-EXTERNAL, which is out of scope).
"""
from __future__ import annotations
import copy, itertools, json, sys
from dataclasses import dataclass, field


UNIT_TYPE = "R"            # the only declared resource type in the fixture
REQUEST_QTY = 3


# ---------------------------------------------------------------- state
@dataclass
class Unit:
    uid: str
    utype: str
    origin: str            # "fixture" | "minted:<run>" | "enacted:<run>"
    encumbered: bool = False


@dataclass
class World:
    holdings: dict          # party -> list[Unit]
    consent_A: bool = True
    fulfilled: set = field(default_factory=set)   # request ids already DELIVERED
    log: list = field(default_factory=list)


    def count(self, p): return len(self.holdings.get(p, []))
    def total(self): return sum(len(v) for v in self.holdings.values())
    def snapshot(self):
        return {p: sorted(u.uid for u in us) for p, us in self.holdings.items()}


def fixture(a_units=10):
    return World(holdings={"A": [Unit(f"A{i:02d}", UNIT_TYPE, "fixture") for i in range(1, a_units+1)],
                           "B": []})


# ---------------------------------------------------------------- request
@dataclass
class Request:
    rid: str
    recipient: str = "B"
    qty: object = REQUEST_QTY      # int, or None if unspecified
    utype: object = UNIT_TYPE      # str, or None if unspecified
    extra_predicates: list = field(default_factory=list)  # additional end-state predicates (for F03)
    interrupt_after: object = None # inject a delivery fault after n units (F01-c)


# ---------------------------------------------------------------- validation gates (shared by all configs)
DECLARED_TYPES = {UNIT_TYPE}


def gate_definition(req):
    if req.qty is None or req.utype is None:
        return "UNDEFINED_TARGET", "quantity or unit type not specified"
    if req.utype not in DECLARED_TYPES:
        return "UNDEFINED_TARGET", f"unit type {req.utype!r} is not declared in the fixture"
    if not isinstance(req.qty, int) or req.qty <= 0:
        return "UNDEFINED_TARGET", "quantity is not a positive integer"
    return None


def gate_coherence(req, w):
    """Detect end-state predicates that cannot all hold under the fixture's DEFINITIONS.
    Definitional identity: total == sum of holdings (true by definition of 'total').
    That identity is not a conservation law; creation may change the total, nothing can make
    total differ from the sum of holdings without redefining 'total'."""
    preds = req.extra_predicates
    fixed = {}
    for kind, party, val in preds:
        if kind == "count":
            if party in fixed and fixed[party] != val:
                return "INCOHERENT_TARGET", f"{party} required to hold both {fixed[party]} and {val}"
            fixed[party] = val
    want_B = w.count(req.recipient) + req.qty
    if req.recipient in fixed and fixed[req.recipient] != want_B:
        return "INCOHERENT_TARGET", f"request requires {req.recipient}={want_B} and {req.recipient}
={fixed[req.recipient]}"
```

```
    for kind, party, val in preds:
        if kind == "total":
            parties = set(w.holdings) | set(fixed)
            known = {p: fixed.get(p) for p in parties}
            if all(v is not None for v in known.values()) or (req.recipient not in fixed):
                # fill recipient from request
                known[req.recipient] = want_B
                if all(v is not None for v in known.values()):
                    s = sum(known.values())
                    if s != val:
                        return "INCOHERENT_TARGET", f"total required {val} but required holdings sum to {s}"
    return None


# ---------------------------------------------------------------- configurations
class Config:
    cid = "BASE"
    capability = "BASE"
    def run(self, w: World, req: Request, run_id: str) -> dict: raise NotImplementedError


def valuation(req):
    # Pilot-only synthetic convention (1 R-unit delivered == 1 valuation unit). NOT a permanent QC rule.
    return {"domain": "synthetic-R-equivalence", "amount": req.qty, "unit": f"{req.utype}-unit",
            "evidence": "E0-STIPULATED convention"}


class Eval(Config):
    cid = "C-EVAL"; capability = "EVALUATE"
    def run(self, w, req, run_id):
        return {"status": "VALUED", "value_object": valuation(req), "delivered": 0}


class Transfer(Config):
    cid = "C-XFER"; capability = "TRANSFER"
    atomic = True
    def run(self, w, req, run_id):
        if not w.consent_A:
            return {"status": "REJECTED_UNAUTHORIZED", "delivered": 0, "reason": "A's consent is not granted"}
        if w.count("A") < req.qty:
            return {"status": "REJECTED_INSUFFICIENT", "delivered": 0,
                    "reason": f"A holds {w.count('A')} < {req.qty}; no other pool exists"}
        moved = []
        for i in range(req.qty):
            if req.interrupt_after is not None and i == req.interrupt_after:
                if self.atomic:
                    for u in moved:  # roll back
                        w.holdings[req.recipient].remove(u); w.holdings["A"].append(u)
                    return {"status": "FAILED_ROLLED_BACK", "delivered": 0,
                            "reason": f"fault after {i} units; atomic rule restored initial state"}
                return {"status": "PARTIAL", "delivered": len(moved),
                        "reason": f"fault after {i} units; non-atomic rule leaves partial state"}
            u = w.holdings["A"].pop(0); w.holdings[req.recipient].append(u); moved.append(u)
        return {"status": "DELIVERED", "delivered": len(moved), "moved_uids": [u.uid for u in moved]}


class TransferNonAtomic(Transfer):
    cid = "C-XFER-NA"; atomic = False


class Create(Config):
    cid = "C-CREATE"; capability = "CREATE"
    def run(self, w, req, run_id):
        made = []
        for i in range(req.qty):
            if req.interrupt_after is not None and i == req.interrupt_after:
                # declared rule: minted units that were not delivered are void; delivered ones remain
                return {"status": "PARTIAL", "delivered": len(made),
                        "reason": f"fault after {i} units; creation is not rolled back under the declared rule"}
            u = Unit(f"M-{run_id}-{i+1}", req.utype, f"minted:{run_id}")
            w.holdings[req.recipient].append(u); made.append(u)
        return {"status": "DELIVERED", "delivered": len(made), "minted_uids": [u.uid for u in made]}


class EnactUnspecified(Config):
    """ENACT with no declared side-effect rule. The packet requires this configuration to
    report UNDERSPECIFIED_CONFIG rather than silently choosing transfer or creation."""
    cid = "C-ENACT-U"; capability = "ENACT-UNSPECIFIED"
    def run(self, w, req, run_id):
        return {"status": "UNDERSPECIFIED_CONFIG", "delivered": 0,
                "reason": "target predicate 'B gains 3' does not determine A, the total, or unit provenance"}


class EnactMinimal(Config):
    """ENACT with a declared minimal-change rule: only variables named in the target predicate
    change; every other variable is frozen. Unit provenance label: enacted."""
    cid = "C-ENACT-M"; capability = "ENACT-MINIMAL"
    def run(self, w, req, run_id):
        made = []
        for i in range(req.qty):
            if req.interrupt_after is not None and i == req.interrupt_after:
                return {"status": "PARTIAL", "delivered": len(made), "reason": "fault injected"}
```

```
            u = Unit(f"E-{run_id}-{i+1}", req.utype, f"enacted:{run_id}")
            w.holdings[req.recipient].append(u); made.append(u)
        return {"status": "DELIVERED", "delivered": len(made), "enacted_uids": [u.uid for u in made]}


class EnactNamedSource(Config):
    """ENACT whose target predicate also names A: 'B gains 3 AND A loses 3'."""
    cid = "C-ENACT-S"; capability = "ENACT-NAMED-SOURCE"
    def run(self, w, req, run_id):
        if w.count("A") < req.qty:
            return {"status": "INCOHERENT_TARGET", "delivered": 0,
                    "reason": f"predicate 'A loses {req.qty}' cannot hold when A holds {w.count('A')} (holdings are
non-negative by definition)"}
        r = Transfer().run(w, req, run_id)
        return r


CONFIGS = [Eval(), Transfer(), TransferNonAtomic(), Create(), EnactUnspecified(), EnactMinimal(), EnactNamedSource()]


# ---------------------------------------------------------------- engine wrapper (shared gates, replay rule)
def execute(config: Config, w: World, req: Request, run_id: str):
    before = {"A": w.count("A"), "B": w.count("B"), "total": w.total()}
    g = gate_definition(req) or gate_coherence(req, w)
    if g:
        res = {"status": g[0], "delivered": 0, "reason": g[1]}
    elif req.rid in w.fulfilled and not isinstance(config, Eval):
        res = {"status": "DUPLICATE_SUPPRESSED", "delivered": 0,
               "reason": f"request {req.rid} already DELIVERED; replay rule: same request id is idempotent"}
    else:
        res = config.run(w, req, run_id)
        if res["status"] == "DELIVERED":
            w.fulfilled.add(req.rid)
    after = {"A": w.count("A"), "B": w.count("B"), "total": w.total()}
    res.update({"config": config.cid, "rid": req.rid, "before": before, "after": after})
    res["completion_predicate"] = completion(before, after, res, req)
    res["verdict_A_internal_consistency"] = check_invariants(config, before, after, res, w)
    res["verdict_B_requested_outcome"] = "SATISFIED" if res["completion_predicate"] else "NOT SATISFIED"
    res["consequence_record"] = "NONE (consequence model not active in Pilot 001; no NCOIN emitted)"
    res["settlement_record"] = "ordinary log entry only (1COIN not active in Pilot 001)"
    w.log.append(res)
    return res


def completion(before, after, res, req):
    """Completion predicate CP-001: B's holdings increased by exactly qty units of the declared
    type, all unencumbered, in this run, with status DELIVERED."""
    return res["status"] == "DELIVERED" and (after["B"] - before["B"]) == (req.qty if isinstance(req.qty, int) else -1)


BIND_BY_LABEL = False   # True reproduces the v0.1-draft defect (invariants bound to config labels)


def _is(config, label_ids, capabilities):
    if BIND_BY_LABEL:
        return config.cid in label_ids
    return config.capability in capabilities


def check_invariants(config, before, after, res, w):
    issues = []
    if after["total"] != after["A"] + after["B"]:
        issues.append("definitional identity total==A+B violated (undeclared party holds units)")
    d = res.get("delivered", 0)
    if res["status"] in ("DELIVERED", "PARTIAL") and after["B"] - before["B"] != d:
        issues.append("reported delivery does not match B's change")
    if res["status"] not in ("DELIVERED", "PARTIAL") and (after != before):
        issues.append("state changed although status reports no delivery")
    if _is(config, ("C-XFER", "C-XFER-NA", "C-ENACT-S"), ("TRANSFER", "ENACT-NAMED-SOURCE")):
        if res["status"] in ("DELIVERED", "PARTIAL") and after["total"] != before["total"]:
            issues.append("transfer changed the total")
        if res["status"] in ("DELIVERED", "PARTIAL") and (before["A"] - after["A"]) != d:
            issues.append("A's debit does not equal B's credit")
    if _is(config, ("C-CREATE", "C-ENACT-M"), ("CREATE", "ENACT-MINIMAL")) and res["status"] in ("DELIVERED",
"PARTIAL"):
        if after["A"] != before["A"]: issues.append("creation/minimal enactment changed A")
        if after["total"] - before["total"] != d: issues.append("total did not rise by the created amount")
    if _is(config, ("C-EVAL",), ("EVALUATE",)) and after != before:
        issues.append("evaluation-only config moved resources")
    return "CONSISTENT" if not issues else "INCONSISTENT: " + "; ".join(issues)


def check_log_idempotency(log):
    """Cross-step assertion R-ID: a request id may reach DELIVERED at most once per world."""
    seen, dup = set(), []
    for e in log:
        if e["status"] == "DELIVERED":
            if e.get("rid") in seen: dup.append(e.get("rid"))
            seen.add(e.get("rid"))
    return "CONSISTENT" if not dup else f"INCONSISTENT: request id(s) delivered more than once: {dup}"
```

```
# ---------------------------------------------------------------- cases
def cases():
    return [
        ("P001-F01-BASE", lambda: fixture(10), lambda: [Request("REQ-1")]),
        ("P001-F01-a (A holds 2)", lambda: fixture(2), lambda: [Request("REQ-1")]),
        ("P001-F01-b (replay same id)", lambda: fixture(10), lambda: [Request("REQ-1"), Request("REQ-1")]),
        ("P001-F01-b2 (same content, new id)", lambda: fixture(10), lambda: [Request("REQ-1"), Request("REQ-2")]),
        ("P001-F01-c (fault after 2)", lambda: fixture(10), lambda: [Request("REQ-1", interrupt_after=2)]),
        ("P001-F01-d (consent revoked)", lambda: (lambda w: (setattr(w, "consent_A", False), w)[1])(fixture(10)),
lambda: [Request("REQ-1")]),
        ("P001-F01-e (unit type unstated)", lambda: fixture(10), lambda: [Request("REQ-1", utype=None)]),
        ("P001-F03-U1 (undeclared type Q)", lambda: fixture(10), lambda: [Request("REQ-1", utype="Q")]),
        ("P001-F03-U2 (quantity unstated)", lambda: fixture(10), lambda: [Request("REQ-1", qty=None)]),
        ("P001-F03-I1 (B=3 and B=0)", lambda: fixture(10), lambda: [Request("REQ-1", extra_predicates=[("count", "B",
0)])]),
        ("P001-F03-I2 (A stays 10, total stays 10)", lambda: fixture(10), lambda: [Request("REQ-1",
extra_predicates=[("count", "A", 10), ("total", None, 10)])]),
    ]


def run_all():
    rows = []
    for (case_id, mkworld, mkreqs) in cases():
        for cfg in CONFIGS:
            w = mkworld()
            reqs = mkreqs()
            outs = []
            for k, r in enumerate(reqs):
                outs.append(execute(cfg, w, r, f"{case_id.split()[0]}-{cfg.cid}-{k+1}"))
            rows.append({"case": case_id, "config": cfg.cid, "steps": outs})
    return rows


# ---------------------------------------------------------------- mutation tests
# Deliberately defective implementations. A useful assertion set must flag each one.
class BugNoDebit(Transfer):
    cid = "MUT-1 transfer-without-debit"
    def run(self, w, req, run_id):
        for i in range(req.qty):
            w.holdings["B"].append(Unit(f"X{i}", req.utype, "fixture"))
        return {"status": "DELIVERED", "delivered": req.qty}


class BugReceiptBeforeDelivery(Transfer):
    cid = "MUT-2 certify-before-delivery"
    def run(self, w, req, run_id):
        # reports DELIVERED, but the fault stops movement after 2 units
        moved = 0
        for i in range(req.qty):
            if req.interrupt_after is not None and i == req.interrupt_after: break
            w.holdings["B"].append(w.holdings["A"].pop(0)); moved += 1
        return {"status": "DELIVERED", "delivered": req.qty}


class BugHiddenPool(Transfer):
    cid = "MUT-3 hidden-third-party-pool"
    def run(self, w, req, run_id):
        if w.count("A") < req.qty:
            w.holdings["POOL"] = []
            for i in range(req.qty):
                w.holdings["B"].append(Unit(f"P{i}", req.utype, "pool"))
            w.holdings["POOL"] = []  # pool silently drained/hidden
            return {"status": "DELIVERED", "delivered": req.qty}
        return super().run(w, req, run_id)


class BugEvalMoves(Eval):
    cid = "MUT-4 evaluation-that-moves"
    def run(self, w, req, run_id):
        w.holdings["B"].append(w.holdings["A"].pop(0))
        return {"status": "VALUED", "value_object": valuation(req), "delivered": 0}


class BugCreateDebits(Create):
    cid = "MUT-5 creation-that-debits-A"
    def run(self, w, req, run_id):
        r = super().run(w, req, run_id)
        w.holdings["A"].pop(0)
        return r


def execute_no_replay_guard(config, w, req, run_id):
    before = {"A": w.count("A"), "B": w.count("B"), "total": w.total()}
    res = config.run(w, req, run_id)
    after = {"A": w.count("A"), "B": w.count("B"), "total": w.total()}
    res.update({"config": config.cid + " (MUT-6 replay-guard removed)", "rid": req.rid, "before": before, "after":
after})
    w.log.append(res)
    res["verdict_A_internal_consistency"] = check_invariants(config, before, after, res, w)
    return res
```

```
def mutation_suite():
    out = []
    def one(label, cfg, world, req, detect):
        r = execute(cfg, world, req, "MUT")
        flagged = detect(r)
        out.append({"mutant": label, "status": r["status"], "after": r["after"],
                    "verdict_A": r["verdict_A_internal_consistency"], "detected": flagged})
    one("MUT-1 transfer-without-debit", BugNoDebit(), fixture(10), Request("REQ-1"),
        lambda r: r["verdict_A_internal_consistency"] != "CONSISTENT")
    one("MUT-2 certify-before-delivery", BugReceiptBeforeDelivery(), fixture(10), Request("REQ-1", interrupt_after=2),
        lambda r: r["verdict_A_internal_consistency"] != "CONSISTENT" or not r["completion_predicate"])
    one("MUT-3 hidden-third-party-pool", BugHiddenPool(), fixture(2), Request("REQ-1"),
        lambda r: r["verdict_A_internal_consistency"] != "CONSISTENT")
    one("MUT-4 evaluation-that-moves", BugEvalMoves(), fixture(10), Request("REQ-1"),
        lambda r: r["verdict_A_internal_consistency"] != "CONSISTENT")
    one("MUT-5 creation-that-debits-A", BugCreateDebits(), fixture(10), Request("REQ-1"),
        lambda r: r["verdict_A_internal_consistency"] != "CONSISTENT")
    # MUT-6: replay guard removed -> second identical request delivers again
    w = fixture(10); t = Transfer()
    r1 = execute_no_replay_guard(t, w, Request("REQ-1"), "M6a")
    r2 = execute_no_replay_guard(t, w, Request("REQ-1"), "M6b")
    idem = check_log_idempotency(w.log)
    dup = idem != "CONSISTENT"
    # detection rule: a request id may reach DELIVERED at most once (CP-001 + replay rule R-ID)
    out.append({"mutant": "MUT-6 replay-guard-removed", "status": f"{r1['status']} then {r2['status']}",
                "after": r2["after"], "verdict_A": r2["verdict_A_internal_consistency"],
                "detected": dup, "log_assertion": idem, "note": "per-step invariants pass; only the cross-step
idempotency assertion detects it"})
    return out


# ------------------------------------------------------------------ M04 numeric check
def m04(B=100.0, alphas=(0.0, 0.25, 0.4, 0.49, 0.5, 0.51, 0.75, 1.0, -0.5), T=150):
    res = []
    for a in alphas:
        fixed = None if abs(1 - 2*a) < 1e-12 else 2*B/(1 - 2*a)
        I = 2*B                       # I_0 = 2B (Gemini's initial condition)
        traj = [I]
        for t in range(1, T+1):
            I = 2*B + 2*a*I
            traj.append(I)
        res.append({"alpha": a, "fixed_point": fixed, "I_1": traj[1], "I_10": traj[10],
                    "I_150": traj[150], "I_149": traj[149]})
    return res


if __name__ == "__main__":
    global_mod = sys.modules[__name__]
    global_mod.BIND_BY_LABEL = True
    muts_label = mutation_suite()
    global_mod.BIND_BY_LABEL = False
    rows = run_all()
    muts = mutation_suite()
    m = m04()
    json.dump({"runs": rows, "mutations_label_bound_draft": muts_label, "mutations": muts, "m04": m},
open("pilot001_reference_results.json", "w"), indent=1, default=str)
    # compact console log
    for r in rows:
        for k, s in enumerate(r["steps"]):
            print(f"{r['case']:<42} {r['config']:<10} step{k+1} {s['status']:<22} A {s['before']['A']}->{s['after']
['A']} "
                  f"B {s['before']['B']}->{s['after']['B']} T {s['before']['total']}->{s['after']['total']} "
                  f"CP={'Y' if s['completion_predicate'] else 'N'} VA={s['verdict_A_internal_consistency'][:12]}")
    print()
    print("MUTATION SUITE - draft harness (invariants bound to config labels)")
    for x in muts_label: print(" ", x["mutant"], "| detected:", x["detected"], "|", x["verdict_A"])
    print("MUTATION SUITE - corrected harness (invariants bound to declared capability)")
    for x in muts: print(" ", x["mutant"], "| detected:", x["detected"], "|", x["verdict_A"], "|",
x.get("log_assertion",""))
    print()
    for x in m: print(x)
```